Comparing Resolution, DP, and DPLL for SAT Solving: A Theoretical and Experimental Study

Vaipan Denis Sebastian
Department of Computer Science
West University Timișoara denis.vaipan05@e-uvt.ro

May 11, 2025

Abstract

This paper presents a comparative study of three fundamental SAT solving techniques: Resolution, Davis-Putnam (DP), and Davis-Putnam-Logemann-Loveland (DPLL). We analyze their theoretical underpinnings, decision-making strategies, and empirical performance on various problem instances. The goal is to assess their relative efficiency and scalability, and to provide insights into their strengths and limitations across different logical complexities. Our findings offer a concise evaluation of the practical differences between these approaches and highlight scenarios where one technique may outperform the others.

Contents

1 Introduction **3**
1.1 Motivation of the Problem. . . . 3
1.2 Informal description of solution 3
1.3 Informal example 3
1.4 Declaration of originality 3
1.5 Paper Organization 3

2 SAT Solving Methods **4**
2.1 Problem Definition 4
2.2 Algorithms 4
2.2.1 Resolution 4
2.2.2 Davis-Putnam(DP) 4
2.2.3 Davis-Putnam-Logemann-Loveland (DPLL) 5

3 Model and Implementation of Problem and Solution **5**
3.1 modeling for satisfiability problems 5
3.1.1 Why python? 5
3.2 modeling for resolution 5
3.3 modeling for Davis-Putnam (DP) 6
3.4 modeling for Davis-Putnam-Logemann-Loveland (DPLL) 6
3.5 implementing a 3-SAT satisfiable generator 6
3.6 Running the program 6
3.7 Code Access 7

4 Case Studies/Experiment **7**
4.1 Datasets 7
4.2 Metrics 7
4.3 Setup 7
4.4 Benchmark 7

5 Results and Discussion **7**
5.1 Discussing the benchmarks 8
5.1.1 early testing, low clause count, From 1 to 10, low variable count 3-5 . . . 8
5.1.2 Clause counts from 10 to 20 9
5.2 Clause counts from 100 and beyond 9
5.3 Scaling the number of variables 10
5.4 The importance of the ratio 12

6 Related Work **12**
6.1 Historical Context 12
6.2 Comparative Studies 12

7 Conclusion and Future Work **12**
7.1 Conclusions 12
7.2 Future Work 12

1 Introduction

1.1 Motivation of the Problem.

SAT (Boolean satisfiability) solving is a foundational problem in computer science. It asks whether there exists an assignment of truth values to variables such that a given propositional formula evaluates to true. SAT is widely known to be NP-complete [2]. As such, numerous algorithms have been developed to solve it efficiently in practice, despite its worst-case complexity.

Among the classic methods are Resolution, a simple but memory and time inefficient approach followed by Davis–Putnam (DP), which improves upon Resolution by integrating simplification strategies. Finally, Davis-Putnam-Logemann-Loveland (DPLL) discards the resolution step entirely in favor of a recursive backtracking technique. Each method has been historically considered an improvement over its predecessor. However, we seek to evaluate if DPLL is truly superior in all practical contexts, or if the older approaches may still hold value under specific circumstances.

1.2 Informal description of solution

To evaluate these methods, we implemented authentic versions of each algorithm in Python, optimizing each of them with our current tool-set, we created a random 3-SAT satisfiable generator in order to perform various comparisons between each method, comparing both from a variable scaling and a clause scaling perspective.

Expectations

We expect that while DPLL will scale better than both DP and resolution on big sets of data (clause/variable wise), DP will likely perform in a more steady manner, with results averaging in between resolution and DPLL, resolution will outperform DP and DPLL in very small instances (very small clause counts), DP will also probably perform better or close to DPLL in low clause count, perhaps the shift occuring around 50 clauses

1.3 Informal example

A set of clauses such as $[(A \vee \neg B),(\neg A \vee B)]$ should be faster with resolution due to the simplicity of the problem, while both DP and DPLL are checking if they can apply different steps on it before solving.

1.4 Declaration of originality

The paper offers original solvers made by the author and offers an experimental comparison of all the implemented solvers, the tests performed will outline edge cases in which, the idea of DPLL always being better is challenged.

1.5 Paper Organization

Section 2 formalizes SAT and the algorithms. Section 3 details our implementation,Section 4 offers the base tools used in creating the benchmarks, Sections 5 present experiments and discuss results. Section 6 contextualizes our findings with prior work, while section 7 offers the key insights taken from the paper while pointing out what has been missed. The work can be read starting from anywhere, but the main point of the paper is on section 5.

2 SAT Solving Methods

2.1 Problem Definition

A **conjunctive normal form (CNF)** formula ϕ is a conjunction of clauses $C_1 \wedge \cdots \wedge C_m$, where each clause C_i is a disjunction of literals (a variable $v \in V$ or its negation $\neg v$). The **Boolean satisfiability problem (SAT)** asks whether there exists an assignment $\sigma \colon V \to \{0, 1\}$ such that for every clause $C_i \in \phi$, at least one literal $\ell \in C_i$ satisfies $\sigma(\ell) = 1$. SAT was proven to be **NP-complete** by Cook in their seminal work [2].

2.2 Algorithms

2.2.1 Resolution

The resolution rule operates on two clauses containing complementary literals. Given clauses $(A \vee \ell)$ and $(\neg\ell \vee B)$, their **resolvent** is $(A \vee B)$. The resolution algorithm proceeds as follows:

1. Generate all possible resolvents from the initial clause set
2. Repeat until either:
   - The empty clause $\perp$ is derived (unsatisfiable)
   - No new resolvents can be generated (satisfiable)

Properties: The method is sound and complete [12], but requires exponential space in the worst case [9].

2.2.2 Davis-Putnam(DP)

The simplification rules outlined by Davis and Putnam [5] are foundational to modern SAT solving. These include procedures for handling unit clauses, pure literals, and resolution-based variable elimination:

- **Unit clause elimination**:
  1. If both p and $\neg p$ appear as single-literal (unit) clauses in a CNF formula F, then F is unsatisfiable.
  2. If only p appears as a unit clause:
     - Remove all clauses containing p
     - Delete all occurrences of $\neg p$ from the remaining clauses
  3. If only $\neg p$ appears as a unit clause:
     - Remove all clauses containing $\neg p$
     - Delete all occurrences of p from the remaining clauses
  4. If the resulting formula is empty (that is, no clauses remain), then the original formula is satisfiable.
- **Pure literal rule**:
  - If a variable p occurs only positively (that is, without negation) or only negatively (that is, always negated), all clauses containing p can be removed
  - This preserves the satisfiability status of the formula
- **Variable elimination via resolution**:

1. Generate all possible resolvents from the initial clause set
2. Repeat until either:
   - The empty clause $\perp$ is derived (unsatisfiable)
   - No new resolvents can be generated (satisfiable)

2.2.3 Davis-Putnam-Logemann-Loveland (DPLL)

DPLL [4] improves upon DP with:

- **Unit propagation**: Immediately assign variables appearing in unit clauses
- **Pure literal elimination**: As in DP
- **Backtracking search**: Recursively try $\phi[v \mapsto 1]$ and $\phi[v \mapsto 0]$

Complexity: DPLL runs in $O(2^n)$ time in the worst case but uses polynomial space [4].

3 Model and Implementation of Problem and Solution

3.1 modeling for satisfiability problems

Due to SAT being NP-Complete [2], it means that there are a variety of heuristics decisions to take and strategies, so for our testing we will be using python as a base programming language, for each method we implemented a script which will test the run time and memory usage.

3.1.1 Why python?

python is a widely used tool for prototyping and data collection in most experiments, it provides the tools necessary to complete the analysis without too many hurdles, it's known as a fast prototyping language for small snippets of code such as ours. Of course, it has its own drawbacks, such as being slower than something like C, or (again compared to C) it relies on an automated garbage collector which can cause unpredictable performance spikes unpredictably [7], on another note, python also dynamically allocates memory which is both a strength and a weakness, it provides a more beginner-friendly experience due to its lack of constraints but it also lowers performance.

Python's garbage collector concerns

It does raise questions as to whether it affects memory measurements, while it does introduce noise in our results, we are following the trend of the results rather than the exact numbers, as our paper concentrates on proving where performance tends to fluctuate between the 3 classic methods.

3.2 modeling for resolution

In order to model for resolution, we took the base rules from theory and applied the concept in Python, we implemented its brute forcing behavior and made minor tweaks which makes our implementation to actively look at the set of clauses to avoid creating an identical or redundant clauses (a subset of another clause that already exists in the set of clauses). Optimizations were made in the inner for loop in order to not check the same clause combination twice, but these optimizations still don't completely remove its weaknesses (i.e. the complexity being $O(n^2)$). There are 2 main checks that abruptly end resolution in order to complete the behavioral table of the other 2 methods, one is giving resolution a maximum number of clauses it can have at

any given moment, as such, once it reaches over 10,000 clauses, we consider that it's running in an infinite loop and returns "UNKNOWN" as while most likely it wouldn't show a result, we didn't practically prove it. The other main check is the number of steps it can take, representing the number of instances that its main function can run for, for the same reason as before but extended to 100.000.

3.3 modeling for Davis-Putnam (DP)

As in the case of modeling resolution, we follow the same principle of applying the basic theory such as unit clause elimination (which removes clauses by enforcing that the respective one literal clause is true which as a result it removes all other clauses containing the literal) and the pure literal rule (which removes all clauses containing the literal if the literal appears as only positive or negative), before going back to resolution. After this basic implementation we expand it further with optimizations, including the optimizations made in the resolution method here while also adding the maximum number of clauses as well, in order to not blur the lines of the method (by adding heuristics) we let it remain like this.

3.4 modeling for Davis-Putnam-Logemann-Loveland (DPLL)

DPLL follows a similar approach to DP (Davis-Putnam) by applying mostly the same first 2 steps but it outperforms it by relying on backtracking instead of pure resolution, it forces assignments on variables in order to find satisfiable outcomes,as a result, rather than creating more clauses, it instead stores a partial solutions to the current set of clauses, the heuristics involved in its implementation is a simple frequency based variable selector in order to target most used literals throughout the sets of clauses, this makes it prioritize speed over memory usage which will be shown in the future tests, especially on low variable count.

3.5 implementing a 3-SAT satisfiable generator

What is 3-SAT?

3-SAT is a satisfiability problem restricted to CNF(conjunctive normal form) where each clause contains exactly 3 literals.

The 3-SAT generator

its a standard 3-SAT generator creating a random satisfiable assignment and then produces clauses that preserve satisfiability, but as with all generators it start producing unsatisfiable results once the number of clauses to number of variables ratio reaches around 4.26 as observed in [3].It also uses the imported tool random [6] in order to create clauses with random variables restricted to the input of the user.

3.6 Running the program

The user is required to just execute the program, they will be asked to enter whether they want to enter in clause range mode or not, this is used in preparing either clause scaling test with a static variable count (by entering y) or the opposite, then based on it they will be asked to input the number of different variables/clauses (the static count) that will be used, the range of the number clauses/variables (the scaling count) to be checked (from start to finish) and the step (incrementation) that will be used throughout, the last mentioned input is useful when using big ranges and variable counts, since resolution tends to slow the program down tremendously, the output will appear in benchmark table.tex and can be inserted directly into latex if needed, users can also manually turn resolution on or off from input.

3.7 Code Access

The full code and instructions are available at:
`https://github.com/SV-Denis/sat-comparison`

4 Case Studies/Experiment

Experimental Design

4.1 Datasets

All of our SAT instances comes from our custom 3-SAT generator, as soon as we start the program we get to choose how many different variables we are using, the range from start to finish for our number of clauses and the step in order to compute even larger sets of data without being slowed down as hard by resolution.

4.2 Metrics

We evaluate memory usage and runtime performance using Python's built-in modules:

- `tracemalloc` [11] to measure peak memory allocation during function execution.
- `timeit` [10] to record the average runtime of each function (tested over 100 runs, configurable via `timeit.timeit(runner, number=runs)`).

Additionally, we track the **clause-to-variable ratio**, as it reveals curious insights regarding the behavior of our SAT generator.

4.3 Setup

Device specifications

- processor:Intel(R) Core(TM) i7-6700K CPU @ 4.00GHz 4.01 GHz
- RAM: 16.0 GB
- Windows: Windows 10 Enterprise (64-bit)
- GPU: NVIDIA GeForce GTX 1080 Ti

The app used for the test is PyCharm version 2024.2.3
The programming language is Python version 3.13.0

4.4 Benchmark

The benchmarks contain the metrics states above, they are made directly in python using imported tools such as tabulate [1] for generating latex booktabs tables. The benchmarks are made to be scalable either by variable count or clause count, making one static while the other dynamic, moving from a range given in the terminal (i.e 1-100).

5 Results and Discussion

Regarding the benchmarks

Due to the sheer size of them, all of them will go inside the repository, we will be discussing only the observations and writing tables only for the sections of the benchmark which show meaningful results

5.1 Discussing the benchmarks

5.1.1 early testing, low clause count, From 1 to 10, low variable count 3-5

Algorithm	Test Case	Variables	Clauses	Time (s)	Memory (KB)	Result	ratio
DPLL	3-sat:1	3	1	0.000004	1.20	True	0.33
DP	3-sat:1	3	1	0.000002	1.10	True	0.33
RES	3-sat:1	3	1	0.000001	0.83	True	0.33
DPLL	3-sat:2	3	2	0.000004	1.38	True	0.67
DP	3-sat:2	3	2	0.000003	1.63	True	0.67
RES	3-sat:2	3	2	0.000003	2.45	True	0.67
DPLL	3-sat:3	3	3	0.000005	1.59	True	1.00
DP	3-sat:3	3	3	0.000003	1.84	True	1.00
RES	3-sat:3	3	3	0.000005	2.67	True	1.00

Our benchmarks reveal an expected but noteworthy pattern: DPLL exhibits higher initial overhead due to its systematic application of propagation rules even for trivial clauses. In contrast, Resolution demonstrates superior performance on small instances ($\leq$4 clauses) owing to its straightforward approach. However, as variable count increases, Resolution's computational and memory requirements grow exponentially, while maintaining its relative advantage only for very low clause counts.

Algorithm	Test Case	Variables	Clauses	Time (s)	Memory (KB)	Result	ratio
DPLL	3-sat:7	3	7	0.000035	4.86	True	2.33
DP	3-sat:7	3	7	0.000026	4.12	True	2.33
RES	3-sat:7	3	7	0.000192	6.36	True	2.33
DPLL	3-sat:8	3	8	0.000028	5.95	True	2.67
DP	3-sat:8	3	8	0.000027	4.34	True	2.67
RES	3-sat:8	3	8	0.000096	5.68	True	2.67
DPLL	3-sat:9	3	9	0.000018	4.38	True	3.00
DP	3-sat:9	3	9	0.000052	4.82	True	3.00
RES	3-sat:9	3	9	0.000288	6.83	True	3.00
DPLL	3-sat:10	3	10	0.000009	3.16	True	3.33
DP	3-sat:10	3	10	0.000016	3.41	True	3.33
RES	3-sat:10	3	10	0.000132	5.72	True	3.33

DP and DPLL both perform at a steady pace at low clause counts, with the increase of their results growing at the same rate, in most cases DP outperforming DPLL in speed while doing worse in memory usage.

5.1.2 Clause counts from 10 to 20

Algorithm	Test Case	Variables	Clauses	Time (s)	Memory (KB)	Result	ratio
DPLL	3-sat:14	3	14	0.000025	5.91	True	4.67
DP	3-sat:14	3	14	0.000118	6.15	True	4.67
RES	3-sat:14	3	14	0.001152	11.28	True	4.67
DPLL	3-sat:15	3	15	0.000024	6.98	True	5.00
DP	3-sat:15	3	15	0.000153	6.15	True	5.00
RES	3-sat:15	3	15	0.000595	8.23	True	5.00
DPLL	3-sat:16	3	16	0.000022	5.65	True	5.33
DP	3-sat:16	3	16	0.000197	6.39	True	5.33
RES	3-sat:16	3	16	0.001194	9.29	True	5.33

DPLL starts to perform the best in terms of speed, outspeeding DP by about 10 times, but due to our aggressive heuristic method, it remains close in terms of memory usage, which is the natural trade off of increasing speed by memorizing the frequency of variables, resolution lacks any meaningful results in this range.

5.2 Clause counts from 100 and beyond

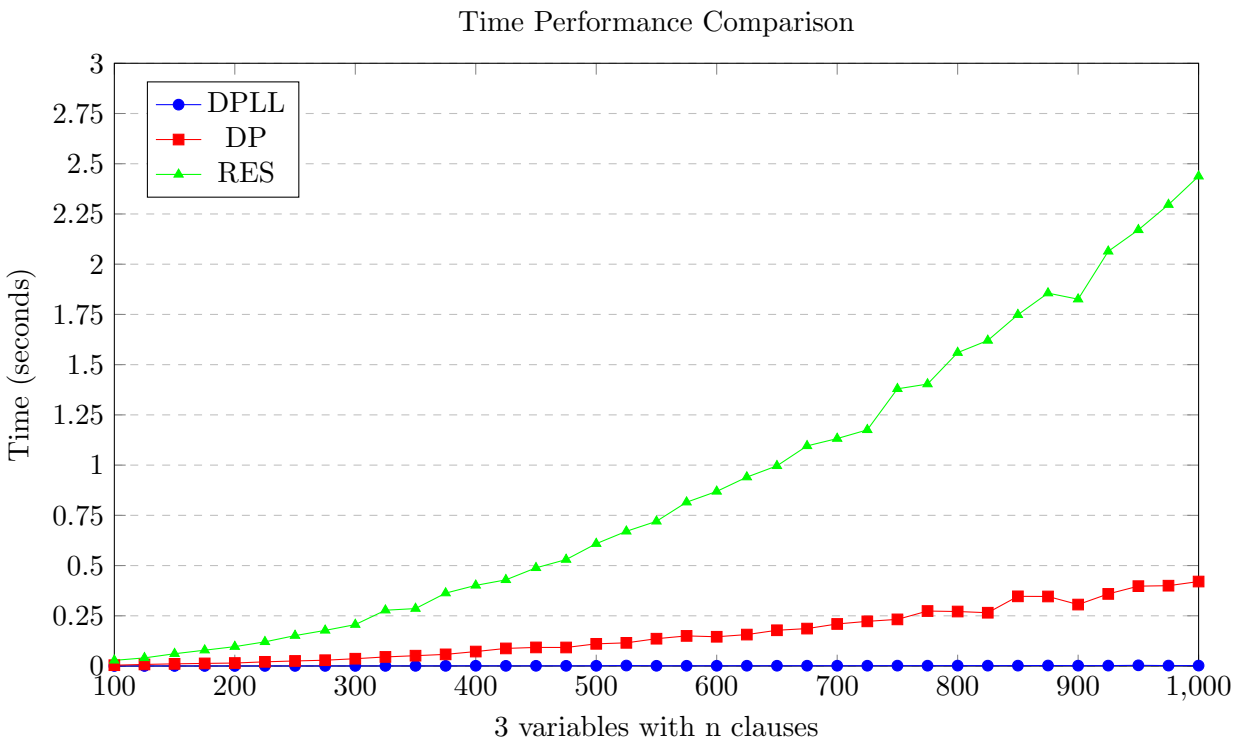

Figure 1: Time performance comparison of DPLL, DP, and RES algorithms across different clause counts while the variable count remains at 3.

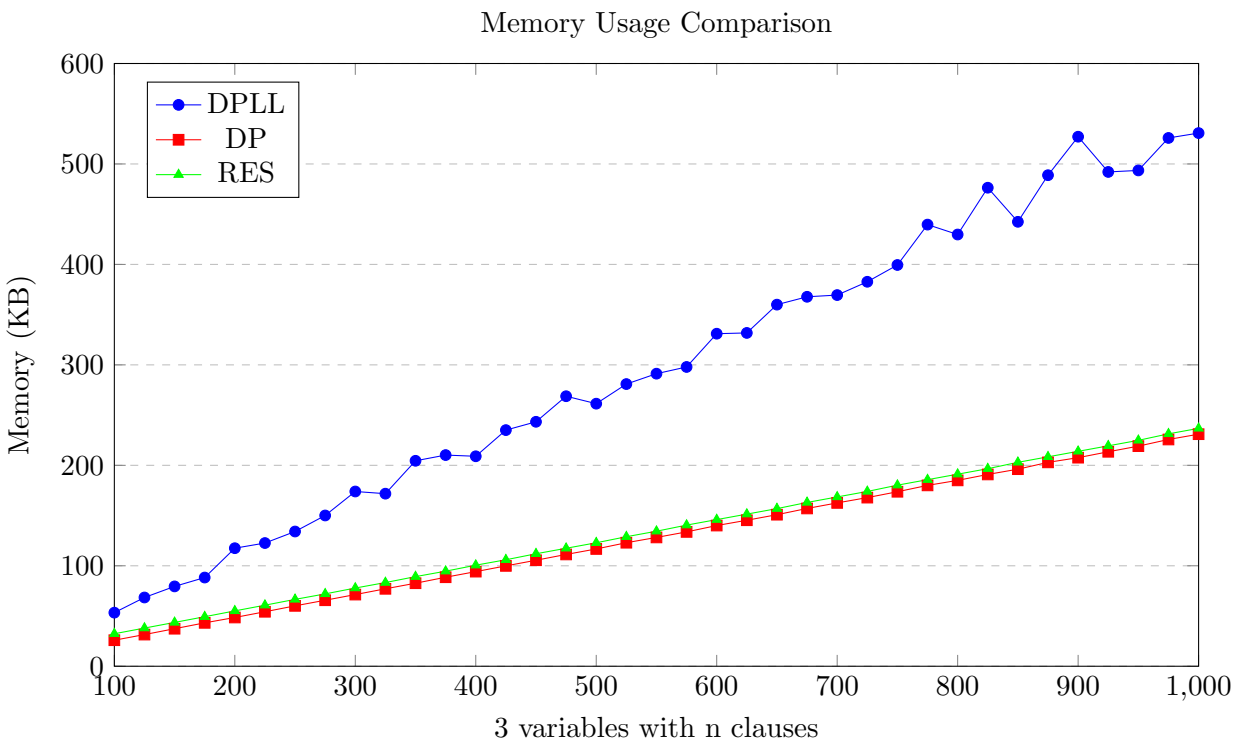

Figure 2: Memory usage comparison of DPLL, DP, and RES algorithms across different clause counts while the variable count remains at 3.

As shown in figure 1 the trends remain the same, Resolution's performance degrades exponentially in terms of speed compared to the other two solutions and in the case of memory (figure 2) both DP and resolution start taking similar amounts of space, resolution exceeding it by a few kilobytes, while DPLL linearly increases as well, further we will provide results by keeping a static clause count in exchange for a dynamic number of variables.

5.3 Scaling the number of variables

Algorithm	Test Case	Variables	Clauses	Time (s)	Memory (KB)	Result	ratio
RES	3-sat:3	3	25	0.003239	15.37	False	8.33
RES	3-sat:4	4	25	0.002210	15.99	True	6.25
RES	3-sat:5	5	25	0.026853	44.98	False	5.00
RES	3-sat:6	6	25	0.028727	44.45	True	4.17
RES	3-sat:7	7	25	0.043987	53.95	True	3.57
RES	3-sat:8	8	25	0.148633	85.15	True	3.12
RES	3-sat:9	9	25	0.101224	92.28	True	2.78
RES	3-sat:10	10	25	0.721151	202.20	True	2.50
RES	3-sat:11	11	25	0.596714	256.96	True	2.27
RES	3-sat:12	12	25	0.210048	172.27	True	2.08
RES	3-sat:13	13	25	1.425701	638.21		1.92
RES	3-sat:14	14	25	1.609831	670.67		1.79

This table presents how fast resolution starts to spike in memory usage when scaling variable count, it was shown alone due to how much it hinders bigger benchmarks, as such, the final 2 graphs will present only the performance between DP and DPLL as they still provide more meaningful data.

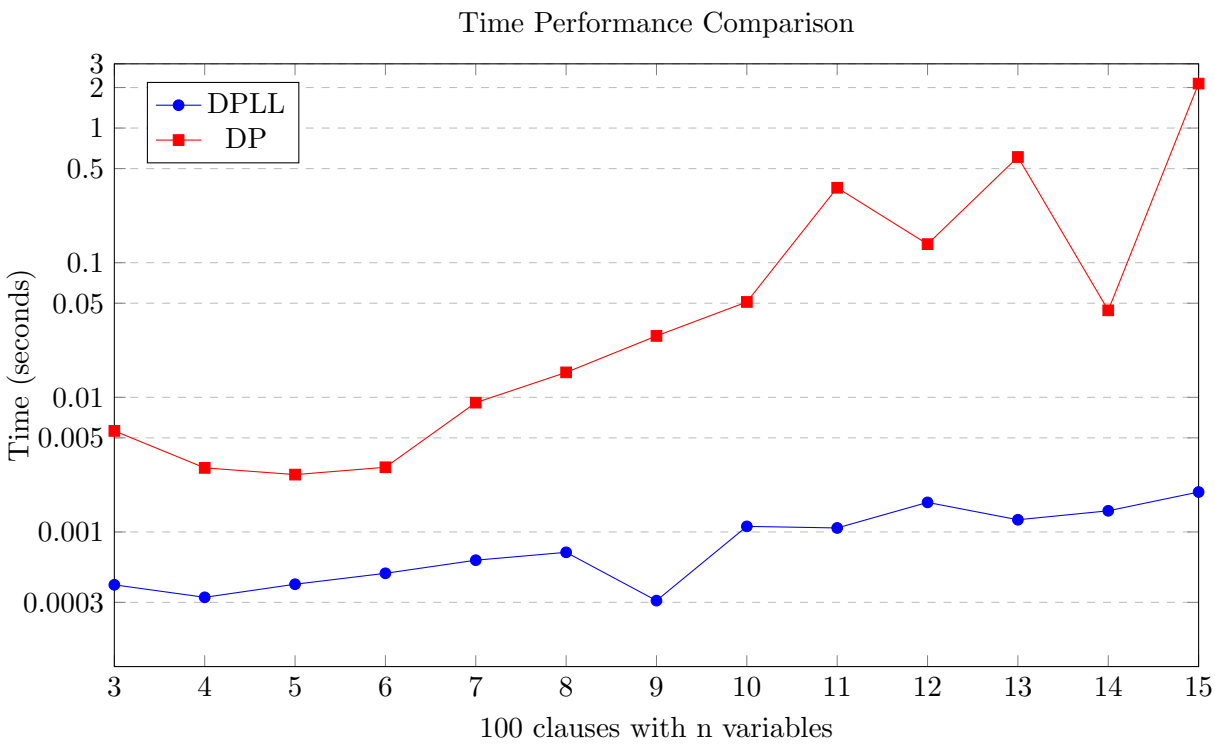

Figure 3: Time performance vs variable count (100 clauses)

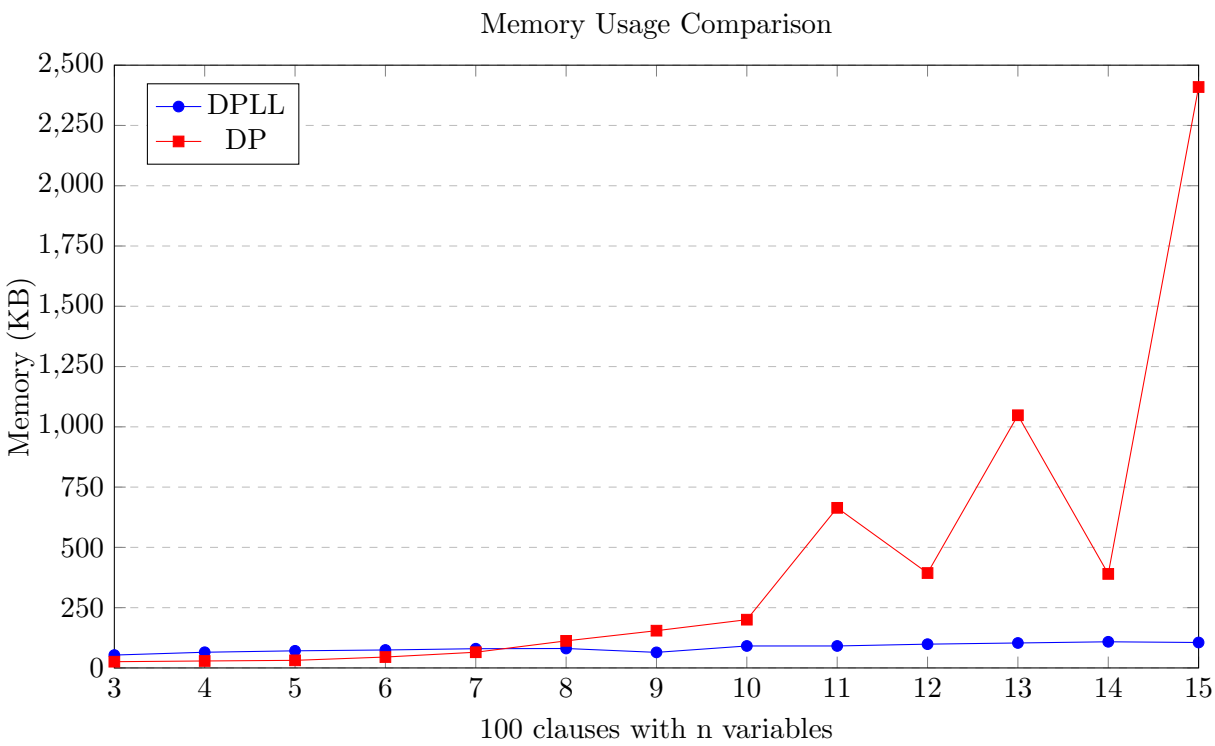

Figure 4: Memory usage vs variable count (100 clauses)

The 2 previous graphs show a different result in the memory department compared to our analysis where we scaled the number of clauses, as seen with resolution earlier, in graph 4 DP's memory usage spikes as a result of the increasing variable count because of the resolution step creating big amounts of extra clauses, while DPLL actively aims to reduce the count of clauses,

our data demonstrates that after 7 variables, the memory usage starts to get exponentially bigger for DP in contrast with the previous results on low variable/clause count, this is due to DP generating unresolvable clauses while DPLL avoids the problem entirely through its backtracking. The first graph3 only further enforces DPLL's speed in contrast with the other methods, clearly showing why DPLL is regarded as a faster and more memory efficient method.

5.4 The importance of the ratio

As the ratio between the number of clauses and the number of variables gets over 4.26 [3] the number of satisfiable sets of clauses starts decreasing, meaning that in the case of resolution it makes it more likely for it to run forever, creating huge amounts of clauses, this will of course affect performance in its instance, hence why in my program, if given a big enough number of variables and clauses, it will abruptly end with the result unknown after a set amount of steps, as such, we can't rely on big variable/clause counts to run tests, instead choosing to make multiple tests on low to medium sized tests.

6 Related Work

6.1 Historical Context

The Davis-Putnam (DP) algorithm [5] introduced variable elimination via resolution, but its memory explosion led to DPLL [4], which replaced resolution with backtracking.

6.2 Comparative Studies

Prior work [4,8] has noted DP's memory inefficiency in large-scale SAT solving, but few studies quantify its performance relative to DPLL on small instances (e.g., < 10 variables). The phase transition analysis of [3] focuses on satisfiability thresholds, not runtime/memory trade-offs. Our work fills this gap by analyzing DP's edge cases where it remains competitive in memory usage.

7 Conclusion and Future Work

7.1 Conclusions

Our experiments reveal a clear trade-off:

- for large instances (¿100 clauses or ¿7 variables) DPLL dominates, with polynomial memory growth ($O(n)$) and stable runtime (fig.3)
- for trivial problems (≤ 4 clauses) Resolution outperforms both DP and DPLL, because of its inherent simplicity
- DP's niche, it maintains memory efficiency below 7 variables (fig 4, but its resolution steps cause exponential scaling ($O(2^n)$) beyond this threshold.

7.2 Future Work

We proved that there are situations in which DPLL gets outperformed in one of 2 categories (speed and memory). The implementation added unnecessary clutter and noise for our benchmarks due to Python's garbage collector, while our results couldn't be as consistent due to random spikes happening in memory, it still proved enough in order to follow the trends of each method in regards to performance, future work could involve, implementing our python code

into c for more clear results,We could further reduce noise by instead implementing more simple code, but regardless, the trends should remain the same.

References

[1] Sergey Astanin. Python tabulate: Pretty-print tabular data in python. Python Package, 2021. Version 0.8.9.

[2] Stephen A. Cook. The complexity of theorem-proving procedures. In *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing*, pages 151–158. ACM, 1971.

[3] James M Crawford and Larry D Auton. Experimental results on the crossover point in random 3-sat. In *AAAI*, volume 96, pages 21–27, 1996.

[4] Martin Davis, George Logemann, and Donald Loveland. A machine program for theorem-proving. *Communications of the ACM*, 5(7):394–397, 1962.

[5] Martin Davis and Hilary Putnam. A computing procedure for quantification theory. *Journal of the ACM*, 7(3):201–215, 1960.

[6] Python Software Foundation. *The Python Standard Library: random*, 2024. Accessed: 2024-05-20.

[7] Python Software Foundation. *Python 3 Memory Management Documentation*, 2025. Sections on generational GC (Section 3).

[8] John Franco and John Martin. A history of satisfiability. In Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors, *Handbook of Satisfiability*, chapter 1, pages 3–55. IOS Press, 2nd edition, 2021.

[9] Armin Haken. The intractability of resolution. *Theoretical Computer Science*, 39:297–308, 1985.

[10] Python Software Foundation. *timeit — Measure execution time of small code snippets*, 2023.

[11] Python Software Foundation. *tracemalloc — Trace memory allocations*, 2023.

[12] John Alan Robinson. A machine-oriented logic based on the resolution principle. *Journal of the ACM*, 12(1):23–41, 1965.